Lab 8: Computing the Sink Tree with Dijkstra's Algorithm

Course: CS353 — Computer Networks (Week 8)
Name: SAYAN SHIT
Roll No: 2401179

1. Network Topology & Adjacency Representation (Part A)

The network consists of $N = 6$ routers labelled **V = {A, B, C, D, E, F}** with undirected links and positive integer costs:

Link	Cost
A – B	2
A – C	5
B – C	1
B – D	4
C – E	3
D – E	1
D – F	2
E – F	6

Adjacency Structure

Each undirected link appears symmetrically in both endpoints' neighbor lists:

- **Router A:** B (cost: 2), C (cost: 5)
- **Router B:** A (cost: 2), C (cost: 1), D (cost: 4)
- **Router C:** A (cost: 5), B (cost: 1), E (cost: 3)
- **Router D:** B (cost: 4), E (cost: 1), F (cost: 2)
- **Router E:** C (cost: 3), D (cost: 1), F (cost: 6)
- **Router F:** D (cost: 2), E (cost: 6)

2. Test Vectors & Forwarding Tables (Parts B & C)

Vector 1: Destination (Sink) = F

- **Cost to F:** A = 8, B = 6, C = 6, D = 2, E = 3, F = 0
- **Next Hop toward F:** A to B, B to D, C to E, D to F, E to D
- **Sink-Tree Edges (node, parent):** (A, B), (B, D), (C, E), (D, F), (E, D)

Forwarding Table for Sink F:

Destination	Next Hop	Cost	Shortest Path
A	B	8	A → B → D → F
B	D	6	B → D → F
C	E	6	C → E → D → F
D	F	2	D → F
E	D	3	E → D → F
F	*(local)*	0	Direct (Sink)

Vector 2: Destination (Sink) = A

- **Cost to A:** A = 0, B = 2, C = 3, D = 6, E = 6, F = 8
- **Next Hop toward A:** B to A, C to B, D to B, E to C, F to D
- **Sink-Tree Edges (node, parent):** (B, A), (C, B), (D, B), (E, C), (F, D)

Forwarding Table for Sink A:

Destination	Next Hop	Cost	Shortest Path
A	*(local)*	0	Direct (Sink)
B	A	2	B → A
C	B	3	C → B → A
D	B	6	D → B → A
E	C	6	E → C → B → A
F	D	8	F → D → B → A

Network Topology, Sink Trees, and Rerouting Visualization

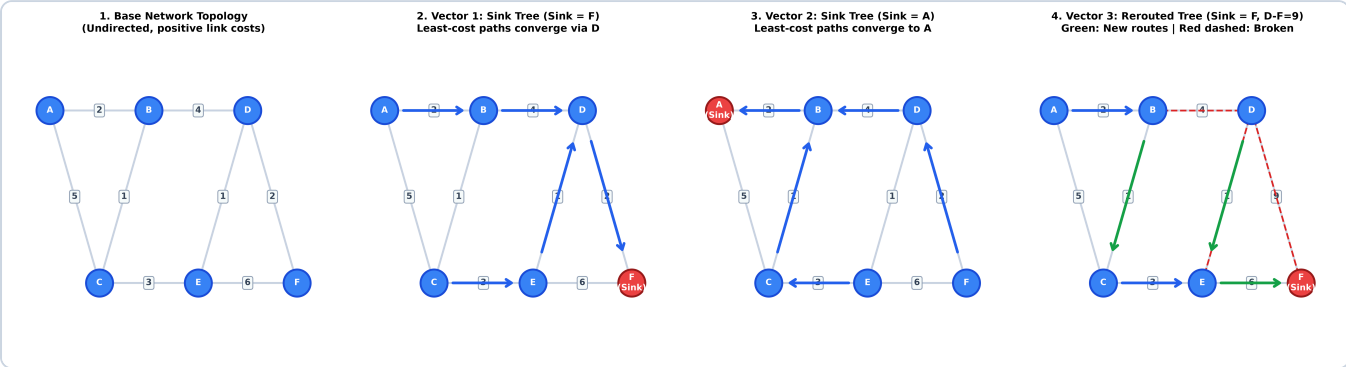


Figure 1: (1) Base Undirected Topology; (2) Vector 1 Sink Tree (Sink F); (3) Vector 2 Sink Tree (Sink A); (4) Vector 3 Rerouted Sink Tree (Sink F, Link D-F=9). Green arrows denote new paths; red dashed denote broken links.

3. Part D: Properties and Rerouting Analysis

Task 1: Verification that the Sink Tree is a Valid Spanning Tree

A subgraph T = (V, E_tree) is a spanning tree of the network topology graph G = (V, E) if and only if it satisfies three fundamental graph-theoretic invariants:

1. **Exactly N − 1 Edges:**
   The network topology contains N = 6 routers (A, B, C, D, E, F). The sink tree assigns exactly one parent edge to each router other than the sink node:

   $$\text{Tree Edges: } |E_{tree}| = N - 1 = 6 - 1 = 5$$

   * For Sink F: Edges are {(A, B), (B, D), (C, E), (D, F), (E, D)} ⟹ 5 edges. * For Sink A: Edges are {(B, A), (C, B), (D, B), (E, C), (F, D)} ⟹ 5 edges.

2. **Reaches Every Router (Full Connectivity):**
   From every router $u \neq$ sink, following the deterministic next-hop pointer sequence:

   $$u \rightarrow \text{parent}(u) \rightarrow \text{parent}(\text{parent}(u)) \rightarrow \ldots \rightarrow \text{sink}$$

   always terminates at the chosen destination (sink). Therefore, every router possesses an active path to the sink, successfully spanning all *N* routers.

3. **Contains No Cycles (Acyclic):**
   Because all link costs are strictly positive ($w > 0$), every relaxation step along a parent pointer strictly decreases the shortest-path distance to the sink:

   $$\text{dist}(u) > \text{dist}(\text{parent}(u))$$

   Because the distance metric decreases monotonically along any path directed toward the sink, a cycle cannot exist (as a cycle would require dist(*u*) > ... > dist(*u*), an impossibility). Furthermore, by graph theory, any connected graph with *N* vertices and *N* − 1 edges is minimally connected and cycle-free.

Hence, the sink tree is rigorously verified to be a valid spanning tree.

Task 2: Modified Topology Rerouting Experiment (Vector 3)

The cost of link **D – F** is raised from **2** to **9**, while all other links and costs remain unchanged.

Recomputed Results for Sink F (Vector 3):

- **Cost to F:** A = 12, B = 10, C = 9, D = 7, E = 6, F = 0
- **Next Hop toward F:** A to B, B to C, C to E, D to E, E to F
- **Sink-Tree Edges (node, parent):** (A, B), (B, C), (C, E), (D, E), (E, F)

Forwarding Table for Sink F (Modified Topology):

Destination	Next Hop	Cost	Shortest Path
A	B	12	A → B → C → E → F
B	C	10	B → C → E → F
C	E	9	C → E → F
D	E	7	D → E → F
E	F	6	E → F
F	*(local)*	0	Direct (Sink)

Tree Edges Comparison (Vector 1 vs. Vector 3):

- **Removed Edges (3):**
  - (B, D)
  - (D, F)
  - (E, D)
- **Added Edges (3):**
  - (B, C)
  - (D, E)
  - (E, F)
- **Unchanged Edges (2):**
  - (A, B)
  - (C, E)

Optimality Principle Explanation:

The **Optimality Principle** states:

> *If router J is on the optimal (least-cost) path from router I to destination K, then the optimal path from J to K also falls along that exact same path.*

Why traffic that previously reached F through D now reaches it through E:

1. **Direct Impact at E and D:**
   In the original topology (Vector 1), router E reached F via D (E → D → F) with cost 1 + 2 = 3, which was lower than its direct link E – F (cost 6). When link D – F increases to 9, path E → D → F costs 1 + 9 = 10. Consequently, the direct link E → F (cost 6) becomes E's new optimal path.
2. **Rerouting of Router D:**
   Router D's direct link to F now costs 9. But D has an alternative route via E: D → E → F with cost 1 + 6 = 7 < 9. Thus, D selects E as its parent.
3. **Upstream Cascading via the Optimality Principle:**
   By the optimality principle, since E lies on the optimal path from D to F, D's optimal route must follow E's optimal route to F (which is E → F).
   For router B:
   - Path via D: B → D → E → F costs 4 + 7 = 11.
   - Path via C: B → C → E → F costs 1 + 3 + 6 = 10.
     Router B therefore chooses C instead of D.
4. **Conclusion:**
   The cost increase on link D – F causes traffic from D, C, B, and A to funnel through router E, replacing tree edges (B, D), (D, F), and (E, D) with (B, C), (D, E), and (E, F).

Task 3: Equivalence on Undirected Graphs & Asymmetric Counterexample

Equivalence Explanation:

On an undirected graph, link costs are symmetric: cost(u, v) = cost(v, u) for every link (u, v). Since the cost of any path is the sum of its link costs, the least-cost path from any router X to destination D has the exact same total cost and vertex sequence as the least-cost path from D to X traversed in reverse. Consequently, running Dijkstra's algorithm once with source D (one-to-all) computes the reverse of the shortest paths from all routers to D (all-to-one), and the predecessor assigned to X by Dijkstra from D is precisely X's next hop toward D in the sink tree.

Situation Where They Differ (Counterexample):

The sink tree to destination D and the Dijkstra source tree from D differ in **directed networks with asymmetric link costs** (such as networks with asymmetric bandwidth/delay like ADSL/cable, or policy-based routing where cost($u \to v$) ≠ cost($v \to u$)).

Concrete Example: Consider a 3-node directed network with nodes D, X, and Y:

- Directed link $D \to X$ with cost 1, but $X \to D$ with cost 10.
- Directed link $D \to Y$ with cost 5, and $Y \to D$ with cost 5.
- Directed link $X \to Y$ with cost 1, and $Y \to X$ with cost 1.

1. **Dijkstra Source Tree from D (One-to-All):**
   - Shortest path $D \to X$: direct link ($D \to X$, cost 1).
   - Shortest path $D \to Y$: via X ($D \to X \to Y$, cost 1 + 1 = 2 < 5).
   - Forward tree edges: ($D \to X$) and ($X \to Y$).
2. **Sink Tree to Destination D (All-to-One):**
   - Shortest path from Y to D: direct link ($Y \to D$, cost 5).
   - Shortest path from X to D: via Y ($X \to Y \to D$, cost 1 + 5 = 6 < 10).
   - Sink tree forwarding edges: X forwards to Y, and Y forwards to D.
   - Sink tree edges: ($X \to Y$) and ($Y \to D$).

In this asymmetric setting, forward Dijkstra from D routes traffic $X \to Y$ to serve destination Y, whereas the sink tree to destination D requires X to route through Y to reach D. To obtain the true sink tree in directed or asymmetric networks, one must run Dijkstra on the *transposed graph* (with all edge directions reversed) starting from destination D.